Waqtuk – Developer work split and shared agreements

Nov 7, 2026 · @Hammad Albalawi

Four developers write the React front-end in parallel. Everybody gets the same kind of work (pages, components, logic and one piece of the app shell), so all four practise HTML, CSS and JavaScript equally. The shared agreements below keep the code from conflicting. Scope: front-end only. The backend is a later phase (about a month away) and will replace only the data layer (`DataContext.jsx` , `context/actions/*` , `context/selectors/*`); pages and components will not change.

1. How the work is split

The rule that prevents conflicts: **every file has exactly one owner. Only the owner edits it.** If you need a change in someone else's file, ask the owner (a comment on the pull request or a short message); they make it.

Dev	Feature area	Pages (22 in total)	UI components	Logic in JavaScript	Shell piece
Dev 1	Activities and reports (time tracking)	Activities, Calendar, Dashboard, Reports, AdminDashboard	Badge, CategoryDot, ProgressBar, BarChart, Toast, ActivityFormModal	`utils/time` , `utils/stats` , activity validator, activity seed, activity actions and selectors	`theme.css` , `global.css` , ThemeContext, ToastContext
Dev 2	Categories, announcements and profiles	Categories, AdminCategories, AdminAnnouncements, MentorAnnouncements, Profile, MentorProfile	Button, Input, Select, Textarea, Checkbox, Radio, Icon, Card, Alert, PageHeader	`DataContext` , `utils/ids` , category validator, category and announcement seeds, their actions and selectors	Sidebar, navConfig, AuthLayout
Dev 3	Goals, feedback and moderation	Goals, Feedback, MentorGoal, MentorStudent, AdminReports	Modal, ConfirmDialog, SegmentedControl, Table	`utils/goals` , goal validator, goal, feedback and report seeds, their actions and selectors	Topbar, AppShell
Dev 4	Accounts and sharing	LoginPage, Sharing, MentorHome, MentorRequests, MentorStudents, AdminUsers	DonutChart, StatCard, EmptyState	account validator, user, sharing and usage seeds, account and sharing actions, their selectors	repository and Vite setup, `main.jsx` , `App.jsx` , `routes.jsx` , AuthContext, route guard, the skeleton

Why this is equal. Every developer writes 5 or 6 pages, 3 to 10 components (the people with few components have the biggest ones: Modal and Table, DonutChart), 7 or 8 logic files and 2 to 5 shell files. Every page mixes the three languages: markup (HTML written as JSX), styling (its own `.css` file) and behaviour (JavaScript with `useState`, events, arrays and objects). The effort of each person is planned to be within about 10 percent of the others. If one person is clearly ahead, the team leader moves one page from the slowest to the fastest; nobody starts a file that has an owner.

Phase 0 (first hour, all four): read this document and agree on it. Dev 4 creates the GitHub repository and pushes the skeleton within about 30 minutes: every file of section 2 as a placeholder (right name, right export), so every import already works. Dev 1 pushes `theme.css` and `global.css` in the first hour. Dev 2 and Dev 3 read the prototype pages of their own area until the skeleton is up.

Phase 1 (about 2 days): everyone writes their components, logic files and shell piece (task sheets in sections 6 to 9), pushing small pieces often.

Phase 2 (about 3 to 4 days): everyone writes their pages and merges each page as soon as it works.

Phase 3 (last 2 days): click through every flow together, compare with the prototype, check mobile and dark mode on every page.

Nothing waits for anything else: from the end of Phase 0 every name that a page imports already exists as a placeholder, and the placeholders get better underneath without changing their props or function signatures.

2. Agreement 1: routes, folders and owners

Stack (the same tools as the course): React with Vite, React Router version 6 (`npm install react-router-dom@6`), plain CSS files with CSS variables, `useState` and `useEffect`, and React Context for the few things every page shares. No backend: the data comes from JSON files in `src/data` and is kept in memory and in `localStorage`. Charts are small hand-written SVG components, no chart library.

```
waqtuk/
  index.html  package.json  .gitignore
  design/                        prototype site, reference only (nobody
edits)
  src/
    main.jsx  App.jsx  routes.jsx                          Dev 4
    styles/      theme.css  global.css                     Dev 1
    layouts/     Sidebar.jsx  navConfig.js  AuthLayout.jsx  Dev 2
                 Topbar.jsx  AppShell.jsx                  Dev 3
    context/     AuthContext.jsx                           Dev 4
```

```
                  ThemeContext.jsx  ToastContext.jsx        Dev 1
                  DataContext.jsx                           Dev 2
      actions/    activities.js                             Dev 1
                  categories.js  announcements.js           Dev 2
                  goals.js  feedback.js                     Dev 3
                  accounts.js  sharing.js                   Dev 4
                  index.js                                  skeleton (export *
from every file)
      selectors/ activities.js                              Dev 1
                  catalog.js                                Dev 2
                  goals.js                                  Dev 3
                  accounts.js                               Dev 4
                  index.js                                  skeleton (export *
from every file)
    components/ui/      index.js (skeleton, nobody edits)  each: Name.jsx +
Name.css
                        Badge CategoryDot ProgressBar Toast            Dev 1
                        Button Input Select Textarea Checkbox Radio
                        Icon Card Alert PageHeader                     Dev 2
                        Modal ConfirmDialog SegmentedControl Table     Dev 3
                        StatCard EmptyState                            Dev 4
    components/charts/  index.js (skeleton)  BarChart (Dev 1)  DonutChart
(Dev 4)
    data/       activities.json                                        Dev 1
                categories.json  announcements.json                    Dev 2
                goals.json  feedback.json  reports.json                Dev 3
                users.json  sharing.json  usage.json                   Dev 4
    utils/      time.js  stats.js                                      Dev 1
                ids.js                                                 Dev 2
                goals.js                                               Dev 3
                validators/  activity.js (Dev 1)  category.js (Dev 2)
                             goal.js (Dev 3)  account.js (Dev 4)
                             index.js
skeleton
    pages/auth/     LoginPage.jsx                                      Dev 4
    pages/student/  Dashboard Activities Categories Goals Calendar
                    Reports Sharing Feedback Profile       owner per section
10
    pages/mentor/   MentorHome MentorRequests MentorStudents
                    MentorStudent MentorGoal
                    MentorAnnouncements MentorProfile      owner per section
10
    pages/admin/    AdminDashboard AdminUsers AdminCategories
                    AdminReports AdminAnnouncements        owner per section
10
```

Each page lives in one file `Name.jsx` plus an optional `Name.css`. A page that needs its own small pieces puts them in a folder next to it, for example `pages/student/activities/ActivityForm.jsx`, and only that page's owner touches that folder.

Path	Page file	Role	Owner
`/`	auth/LoginPage	all	Dev 4
`/dashboard`	student/Dashboard	student	Dev 1
`/activities`	student/Activities	student	Dev 1
`/categories`	student/Categories	student	Dev 2
`/goals`	student/Goals	student	Dev 3
`/calendar`	student/Calendar	student	Dev 1
`/reports`	student/Reports	student	Dev 1
`/sharing`	student/Sharing	student	Dev 4
`/feedback`	student/Feedback	student	Dev 3
`/profile`	student/Profile	student	Dev 2
`/mentor`	mentor/MentorHome	mentor	Dev 4
`/mentor/requests`	mentor/MentorRequests	mentor	Dev 4
`/mentor/students`	mentor/MentorStudents	mentor	Dev 4
`/mentor/students/:id`	mentor/MentorStudent	mentor	Dev 3
`/mentor/goals/:id`	mentor/MentorGoal	mentor	Dev 3
`/mentor/announcements`	mentor/MentorAnnouncements	mentor	Dev 2
`/mentor/profile`	mentor/MentorProfile	mentor	Dev 2
`/admin`	admin/AdminDashboard	admin	Dev 1
`/admin/users`	admin/AdminUsers	admin	Dev 4
`/admin/categories`	admin/AdminCategories	admin	Dev 2
`/admin/reports`	admin/AdminReports	admin	Dev 3
`/admin/announcements`	admin/AdminAnnouncements	admin	Dev 2

`routes.jsx` is written once by Dev 4 in Phase 0 with all 22 routes already pointing to stub pages (each stub is a file that shows only the page title). After that nobody edits `routes.jsx`. Each owner just replaces the content of their own page file.

The sidebar lists per role live in `layouts/navConfig.js` (Dev 2). Student: Dashboard, Activities, Calendar, Reports, Goals, Categories, Sharing, Mentor Feedback, Announcements (opens `/profile#announcements`), Profile. Mentor: Home, Sharing Requests, Shared Students, Announcements, Profile. Admin: Dashboard, Users, Default Categories, Reported Content, System Usage (opens `/admin#usage`), Announcements.

3. Agreement 2: design tokens and CSS rules

Dev 1 copies these variables into `src/styles/theme.css` exactly as written (they come from the prototype, so colours match the design). The top element of the app has the class `wq-shell`, and `wq-dark` is added to it for dark mode. Nobody else defines colours.

```
.wq-shell{--bg:#FFFFFF;--surface:#F7F8FA;--surface-2:#F2F3F6;--
border:#ECECEF;--text:#1C1C1C;--muted:#5F6370;--muted-card:#4A5260;--
faint:#B8BCC8;--track:#E3E5EA;--blue:#EAF2FD;--blue-text:#1F4E8C;--
lav:#EEEFFC;--lav-text:#3E4BC9;--primary:#1C1C1C;--on-primary:#FFFFFF;--
pill:#FFFFFF;--good:#1F6B47;--good-bg:#FFFFFF;--ok:#1F6B47;--ok-bg:#E6F4EC;--
ok-solid:#2E9E6B;--err:#B3261E;--err-bg:#FDECEA;--warn:#8A5A00;--warn-
bg:#FFF4DB;--danger:#C62828;--on-danger:#FFFFFF}
.wq-shell.wq-dark{--bg:#141416;--surface:#1E1F23;--surface-2:#2A2B30;--
border:#2C2D33;--text:#F4F4F6;--muted:#A3A6B0;--muted-card:#B5B9C3;--
faint:#5C5F69;--track:#33353C;--blue:#1B2A40;--blue-text:#9CC3F5;--
lav:#24264A;--lav-text:#B3BAFF;--primary:#FFFFFF;--on-primary:#141416;--
pill:#2E3036;--good:#6FD3A2;--good-bg:#14301F;--ok:#6FD3A2;--ok-bg:#14301F;--
ok-solid:#4FC48E;--err:#F5A9A3;--err-bg:#3A1B19;--warn:#F2C46B;--warn-
bg:#3A2E10;--danger:#F0847C;--on-danger:#1A0B0A}
```

Rule	Agreement
Colours	Only `var(--name)` from the list above. No hex colours in components or pages. The only exceptions are the category colours (below) and the focus outline `#5B6CF0`.
Category colours	Study `#5B6CF0`, Work `#F2B84B`, Exercise `#3FB27F`, Sleep `#A78BDA`, Reading `#8FB3D9`, Social `#E58FB0`, Gaming `#6CC3D6`, Commute `#9AA0A6`. They live in `data/categories.json` (field `color`) and nowhere else.

Rule	Agreement
Font	Inter (Google Fonts link in `index.html`). Page title 24px/600, card title 14px/600, body 14px/400, caption 12px/500.
Shape	Cards 16px radius. Buttons and inputs 10px radius, 44px tall (40px for small buttons). Spacing steps 8, 12, 16, 24, 32 only.
Layout	Sidebar 240px, top bar 64px, page content max width 1160px, page padding 32px.
Mobile	One breakpoint: `max-width: 820px`. Sidebar becomes a strip on top, two-column grids become one column, page padding 20px. Dev 1 provides the utility classes `.wq-g` (grid that collapses), `.wq-hide-sm` and `.wq-main-pad`. Pages use these classes instead of writing their own media queries.
CSS files	One CSS file per component or page, same name as the JSX file, imported at the top of that JSX file.
Class names	Prefix `wq-` then the component then the part, lower case with dashes: `wq-modal`, `wq-modal-title`, `wq-activity-row`. A page prefixes its own classes with the page name (`wq-goals-card`). This stops two people from ever using the same class name.
Inline styles	Only for values that come from data (a category colour, a bar width). Everything else goes in the CSS file.
Dark mode	Never write a dark-mode rule in a component. If everything uses the variables, dark mode works by itself.
Focus	Keep the visible focus outline from `global.css`. Do not remove it.
Icons	Inline SVG, 18px, stroke 1.8, `currentColor`. Copy the paths from the prototype pages (view source). Dev 2 puts them in `components/ui/Icon.jsx`; pages use `<Icon name="plus" />`.

4. Agreement 3: data shapes and the DataContext API

Every list lives in a JSON file in `src/data/` (one file per list, section 4.1) and is loaded by `DataContext` (Dev 2). All durations are whole **minutes**. Dates are text: `'2026-11-07'` for a date, `'2026-11-07T09:00'` for a date and time. Weeks run Monday to Sunday. "Today" in the app is fixed to `2026-11-07` (a constant `TODAY` in `utils/time.js`) so the numbers always match the prototype. **Pages never read the JSON files or `localStorage` themselves**: they only call `useData()`. That is what lets the backend phase replace the data layer without touching a page.

```
// shapes (comments show an example)
User         { id:'s1', name:'Hammad Albalawi', mentorName:'Student A',
email:'student@example.com',
               role:'student' | 'mentor' | 'admin', status:'active' |
'suspended' }
Category     { id:'cat-study', name:'Study', color:'#5B6CF0', type:'default'
| 'custom',
               active:true, ownerId:null }              // ownerId = student
id for custom ones
Activity     { id:'a1', studentId:'s1', categoryId:'cat-study',
               start:'2026-11-07T09:00', end:'2026-11-07T10:30',
notes:'Lecture review' }
Goal         { id:'g1', studentId:'s1', categoryId:'cat-study',
weekStart:'2026-11-02',
               targetMinutes:600, direction:'atLeast' | 'atMost' }
SharingRequest { id:'r1', studentId:'s1', mentorId:'m1', from:'2026-11-02',
to:'2026-11-08',
               scope:['summary','reports'], goalIds:['g1'],
               status:'pending' | 'accepted' | 'declined' | 'cancelled' |
'revoked' | 'ended' }
Feedback     { id:'f1', goalId:'g1', mentorId:'m1', type:'comment' |
'suggestion' | 'flag',
               text:'Study hours look steady.', date:'2026-11-05',
reported:false }
Announcement { id:'n1', title:'Final exams schedule published', body:'...',
               audience:'students' | 'mentors' | 'all', status:'draft' |
'published' | 'unpublished',
               date:'2026-11-05' }
ContentReport { id:'c1', feedbackId:'f3', reporterId:'s1',
reason:'Inappropriate language',
               details:'', status:'pending' | 'dismissed' | 'removed',
decisionReason:'' }
```

Seed data that must match the prototype (each developer writes the seed files of their own feature, table 4.1; every page reads them):

Data	Required content
Users	Student `s1` Hammad Albalawi (mentors see him as "Student A"); students `s2` Student B, `s3` Student C, `s4` Student D; mentors `m1` Mentor A, `m2` Mentor B, `m3` Mentor C; admin `a1` Admin User; `s5` Student E (suspended).
Categories	Default, active: Study, Work, Exercise, Sleep. Default, inactive: Commute. Custom for `s1`: Reading, Social, Gaming.
Activities of `s1`	17 in total. These five must exist exactly: Sat 7 Nov 09:00–10:30 Study "Lecture review"; Sat 7 Nov 11:00–12:00 Work "Project task"; Fri 6 Nov 13:00–14:00 Exercise "Gym"; Thu 5 Nov 23:00–02:00 (ends Fri 6 Nov) Sleep "Overnight"; Thu 5 Nov 16:00–17:00 Study "Reading". Dev 1 adds the rest so the totals below come out exactly.
Totals Mon 2 – Sun 8 Nov	Study 360, Work 510, Exercise 330, Sleep 180, Social 300, Gaming 360 minutes. Total 2040 (34h 00m).
Totals Sat 7 Nov only	Study 90, Work 60 (2h 30m). Previous day (Fri 6 Nov) 120.
Totals 1 – 7 Nov (month so far)	Study 420, Work 540, Exercise 330, Sleep 210, Social 300, Gaming 360 (36h 00m). Same days of October: 1980. Previous week (26 Oct – 1 Nov): 1840.
Goals of `s1`, week of 2 Nov	Study at least 600 (logged 360: remaining 4h), Exercise at least 300 (330: reached), Social at most 480 (300: allowance remaining 3h), Gaming at most 240 (360: exceeded by 2h). Dashboard shows "3 of 4 on track" (Study, Exercise, Social).
Sharing	`s1` to `m2`: pending, 2 Nov – 8 Nov, summary and reports, goal Study. `s1` to `m1`: accepted, same range and scope. For mentor `m1`: pending requests from Student A and Student D; accepted students Student A, B, C.

Data	Required content
Feedback on goal Study (`g1`)	Mentor A: comment "Steady progress this week." (6 Nov), suggestion "Consider an earlier study start." (5 Nov), comment "Nice goal consistency." (2 Nov); Mentor B: flag "Workload looks unbalanced." (4 Nov).
Announcements	Published: "Final exams schedule published" (5 Nov), "Library hours extended" (1 Nov), "Maintenance on Friday night" (28 Oct) for students; "Announcement A" (5 Nov), "Announcement B" (29 Oct), "Announcement C" (20 Oct) for mentors. Admin list also has a draft and an unpublished item.
Content reports	Three pending: mentor comment (inappropriate language), mentor suggestion (harassment), mentor flag (spam), all submitted 6 Nov.

Seed additions (decided from reading every prototype page; same rules as the table above).

- **Rule when the prototype contradicts itself:** the seed below wins, because in the real app every page reads the same records. Known cases: Student A cannot be both pending and accepted with the same mentor; Student B cannot be both suspended and accepted; the student sees the mentor comments with different wording on two pages; the mentor lists show `student.a@example.com` while the student signs in with `student@example.com`.
- **Two extra optional fields:** `SharingRequest.decidedOn` (`'2026-10-28'` , empty while pending) and `Feedback.detail` (long text; the list shows `text` , the detail panel shows `detail ?? text`). Only feedback `f2` has a `detail` : "Consider moving some study hours earlier in the week to reduce late-day load."
- **Users and emails.** The signed-in seed accounts are `s1` (`student@example.com`), `m1` Mentor A (`mentor@example.com`), `a1` (`admin@example.com`). The others: `s2` to `s6` use `student.b@example.com` to `student.f@example.com` (names Student B to Student F, `mentorName` the same), `m2` `mentor.b@example.com` , `m3` `mentor.c@example.com` . Student E (`s5`) is the suspended one. Student F (`s6`) has one Study goal, at least 10 hours, in the week of 2 Nov.
- **Sharing requests (all for 2 Nov – 8 Nov unless stated):** `r1` s1 to m1 accepted, summary and reports, goal Study (`g1`). `r2` s1 to m2 pending, summary and reports, goal Study. `r3` s2 to m1 accepted, 26 Oct – 1 Nov, summary and reports, goals Study and Exercise, decided 2026-10-28. `r4` s3 to m1 accepted, summary only, no goals,

decided 2026-10-26. `r5` s5 to m1 declined, reports only, decided 2026-10-24. `r6` s4 to m1 pending, summary only, no goals. `r7` s6 to m1 pending, summary and reports, goal Study of s6. So Mentor A has 2 pending (Student D, Student F), 3 accepted (A, B, C), 1 declined (E), and shared goals 1 + 2 + 0 = 3.

- **Reading:** exactly 4 activities (dated 26 to 31 October, so the week and month totals do not change) and exactly 1 goal (week of 26 Oct). The category delete dialog says "used by 4 activities and 1 goal".
- **Content reports:** `c1` reports `f1` (comment of Mentor A, 6 Nov) by Student A, reason Inappropriate language, details "Added by the reporter", submitted 6 Nov 2026. `c2` reports a suggestion of Mentor A on a goal of Student B, reason Harassment. `c3` reports a flag of Mentor B on a goal of Student C, reason Spam. All three pending, submitted 6 Nov 2026. Dev 3 creates the two extra goals and the two extra feedback records with any short text. `f1` has `reported: true`.
- **Admin numbers** (dashboard cards for the default range): new students 14, new mentors 3, activities 412, sharing requests 27, moderation reports 5. These are constants in `data/usage.json`, not computed from the seed, and `usageFor(from, to)` in `selectors/accounts.js` reads them. A different date range scales them (see section 10).

What a page may do with data: read it with `useData()` and call the actions below. Pages never edit the arrays directly and never import from `src/data/*` themselves.

Every action returns `{ ok: true }` or `{ ok: false, error: '...' }`. The error text is the exact message from the prototype's states page, so a page only shows it: `if (!res.ok) setError(res.error)`.

Action	Used by	Errors it can return
`addActivity(a)`, `updateActivity(id, changes)`, `deleteActivity(id)`	Activities, Calendar	missing fields, end before start, overlaps another activity, in the future
`addCategory(name)`, `renameCategory(id, name)`, `deleteCategory(id, reassignToId)`	Categories	empty name, duplicate name, still used (needs reassign)
`addGoal(g)`, `updateGoal(id, changes)`, `deleteGoal(id)`	Goals	duplicate goal for category and week, target not between 1 and 168 hours

Action	Used by	Errors it can return
`sendRequest(r)`, `cancelRequest(id)`, `revokeAccess(id)`	Sharing	end date before start, request for this mentor already exists
`acceptRequest(id)`, `declineRequest(id)`, `endSharing(id)`	MentorHome, MentorRequests, MentorStudents	request no longer available
`addFeedback({goalId, type, text})`, `reportFeedback(id, reason, details)`	MentorGoal, Feedback	empty text, access withdrawn, already reported
`updateProfile(changes)`	Profile, MentorProfile	name or email missing, invalid email, email already used
`register(data)`, `login(email, password)`	Login	missing fields, invalid email or password, account suspended, email already registered, passwords do not match, password shorter than 8
`updateUser(id, changes)`, `setUserStatus(id, status, reason)`	AdminUsers	invalid email, duplicate email, reason required, status already set
`addDefaultCategory(name)`, `renameDefaultCategory(id, name)`, `setCategoryActive(id, active)`	AdminCategories	empty name, duplicate name
`decideReport(id, 'dismissed' | 'removed', reason)`	AdminReports	reason required, already handled, content already removed
`saveAnnouncement(a)`, `publishAnnouncement(id)`, `unpublishAnnouncement(id)`	AdminAnnouncements	title or message missing, invalid audience

Data it exposes: `currentUser`, `users`, `categories`, `activities`, `goals`, `sharingRequests`, `feedback`, `announcements`, `contentReports`. If a page needs an action or a value that is not in this table, its owner asks the owner of the matching file in table 4.1 in a pull request comment, and that person adds it and updates the tables.

4.1 Who writes which seed file, action and selector

The data layer is split by feature, so every developer writes some JavaScript logic and nobody waits for one "data person". `DataContext.jsx` (Dev 2) is only the wiring: it loads all JSON files, holds them in one `useState`, saves them to `localStorage` under `waqtuk.data.v1`, and adds every function exported by `context/actions/*` and `context/selectors/*` to what `useData()` returns. **Adding an action or a selector means adding a function to your own file; nobody edits `DataContext.jsx` or the `index.js` files.**

Dev	Seed files in `data/`	Actions in `context/actions/`	Selectors in `context/selectors/`
Dev 1	`activities.json`	`activities.js`: `addActivity`, `updateActivity`, `deleteActivity`	`activities.js`: `myActivities`
Dev 2	`categories.json`, `announcements.json`	`categories.js`: `addCategory`, `renameCategory`, `deleteCategory`, `addDefaultCategory`, `renameDefaultCategory`, `setCategoryActive`. `announcements.js`: `saveAnnouncement`, `publishAnnouncement`, `unpublishAnnouncement`	`catalog.js`: `getCategory`, `myCategories`, `announcementsFor`
Dev 3	`goals.json`, `feedback.json`, `reports.json`	`goals.js`: `addGoal`, `updateGoal`, `deleteGoal`. `feedback.js`: `addFeedback`, `reportFeedback`, `decideReport`	`goals.js`: `myGoals`, `myFeedback`

Dev	Seed files in `data/`	Actions in `context/actions/`	Selectors in `context/selectors/`
Dev 4	`users.json`, `sharing.json`, `usage.json`	`accounts.js`: `register`, `login`, `updateProfile`, `updateUser`, `setUserStatus`. `sharing.js`: `sendRequest`, `cancelRequest`, `revokeAccess`, `acceptRequest`, `declineRequest`, `endSharing`	`accounts.js`: `getUser`, `myRequests`, `mentorRequests`, `sharedStudents`, `getSharedData`, `usageFor`

`DataContext` itself provides the plain lists, `currentUser` (always `users.find(u => u.id === useAuth().userId)`) and `resetData()` (restores the seed; used only by a hidden button on the `/components` preview route). Ids of new records come from `makeId(prefix)` in `utils/ids.js` (Dev 2), for example `makeId('a')`.

Every action file looks like this (a pure function: it receives the data, returns the new data and a result, and never changes what it received):

```
// context/actions/activities.js
import { makeId } from '../../utils/ids';
import { validateActivity } from '../../utils/validators';

export function addActivity(data, payload, ctx) {          // ctx = {
currentUser }
  const check = validateActivity(payload, { activities: data.activities,
categories: data.categories });
  if (!check.ok) return { data, result: check };            // error: nothing
changes
  const activity = { id: makeId('a'), studentId: ctx.currentUser.id,
...payload };
  return { data: { ...data, activities: [...data.activities, activity] },
result: { ok: true, id: activity.id } };
}
```

Every selector file looks like this (receives the data, returns a value; a selector that needs an argument returns a function):

```
// context/selectors/activities.js
export function myActivities(data, ctx) {
  return data.activities
    .filter(a => a.studentId === ctx.currentUser?.id)
    .sort((a, b) => b.start.localeCompare(a.start));        // newest first
}
export function getActivity(data) {
  return (id) => data.activities.find(a => a.id === id);   // used as
getActivity(id)
}
```

How `DataContext` wires them (Dev 2; the same few lines for every action and selector, so adding one never needs an edit here):

```
const [data, setData] = useState(loadFromStorageOrSeed);
useEffect(() => { localStorage.setItem('waqtuk.data.v1',
JSON.stringify(data)); }, [data]);
const ctx = { currentUser: data.users.find(u => u.id === userId) };
// for each exported action: call it, store the new data when it changed,
return its result
const actions = Object.fromEntries(Object.entries(allActions).map(([name,
fn]) => [name, (payload) => {
  const out = fn(data, payload, ctx);
  if (out.data !== data) setData(out.data);
  return out.result;
}]));
// for each exported selector: run it on the current data
const selectors = Object.fromEntries(Object.entries(allSelectors).map(([name,
fn]) => [name, fn(data, ctx)]));
```

One action call per user click. Seed files are plain JSON arrays, for example `activities.json` is `[ { "id": "a1", "studentId": "s1", "categoryId": "cat-study", "start": "2026-11-07T09:00", "end": "2026-11-07T10:30", "notes": "Lecture review" } ]`, and `usage.json` is one object `{ "students": 14, "mentors": 3, "activities": 412, "requests": 27, "reports": 5 }` for a 7 day range.

4.2 Selector return values

Selectors are part of the contract. `useData()` returns them, so pages never filter the raw lists themselves. A selector marked (fn) is used as a function, for example `getUser(id)`.

Selector	Owner	Returns
`getUser(id)` (fn)	Dev 4	the user record or `undefined`

Selector	Owner	Returns
`getCategory(id)` (fn)	Dev 2	the category record or `undefined`
`getActivity(id)` (fn)	Dev 1	the activity record or `undefined`
`myCategories`	Dev 2	categories the signed-in student can use: active default ones in fixed order (Study, Work, Exercise, Sleep) then their own custom ones
`myActivities`	Dev 1	activities of the signed-in student, newest start first
`myGoals`	Dev 3	goals of the signed-in student
`myFeedback`	Dev 3	feedback on goals of the signed-in student, newest first
`myRequests`	Dev 4	sharing requests of the signed-in student
`mentorRequests`	Dev 4	requests addressed to the signed-in mentor
`sharedStudents`	Dev 4	`[{ request, student }]` for the signed-in mentor, accepted requests only
`getSharedData(requestId)` (fn)	Dev 4	`{ student, activities, goals }` already limited by the access rule below
`announcementsFor(role)` (fn)	Dev 2	published announcements whose audience is that role or all, newest first
`usageFor(from, to)` (fn)	Dev 4	the usage numbers of that range for the administrator dashboard

Access rule (lives in Dev 4's `selectors/accounts.js`): a mentor can read a student's activities and goals only through an accepted request, only inside its `from` and `to` dates, and only the goals in its `goalIds`. `sharedStudents` and `getSharedData(requestId)` enforce this. `MentorStudent` and `MentorGoal` (Dev 3) use them and never read activities directly.

5. Agreement 4: contexts, UI components, charts and helper functions

This section is the contract. Names, props and return values below are exact. A page imports only what is listed here. Every file has one owner (the tree in section 2 and the tables below show who), and every developer owns some components, some logic and

some shell, so everybody works in HTML, CSS and JavaScript from the first day. Nobody renames or removes anything in this section without the team leader's approval.

Rule for all visible text. Every label, button, message, empty state and heading on screen is copied letter for letter from the prototype page in `design/`. The example strings in this document only show the shape. If the prototype and this document disagree on a text, the prototype wins, except for the items of the list at the top of section 10, where this document wins.

5.1 Import rules

```
import { Button, Input, Modal } from '../../components/ui';      // never
from a component's own file
import { DonutChart, BarChart } from '../../components/charts';
import { useData } from '../../context/DataContext';
import { useAuth } from '../../context/AuthContext';
import { useToast } from '../../context/ToastContext';
import { useTheme } from '../../context/ThemeContext';
import { formatDuration, getPeriodRange } from '../../utils/time';
import { sumByCategory } from '../../utils/stats';
import { goalProgress, goalMessage } from '../../utils/goals';
import { validateActivity, MESSAGES } from '../../utils/validators';   // the
folder's index.js
```

Provider order in `main.jsx` (Dev 4): `BrowserRouter > ThemeProvider > ToastProvider > AuthProvider > DataProvider > App`.

5.2 Contexts

Hook	Owner	Returns
`useAuth()`	Dev 4	`{ userId, role, isAuthenticated, signIn(userId), signOut() }`. `role` is `'student'`, `'mentor'`, `'admin'` or `null`. The session is kept in `localStorage` under `waqtuk.session`. `AuthContext` stores only the id and the role, never the user object.
`useTheme()`	Dev 1	`{ theme, toggleTheme(), setTheme(t), themeClass }`. `theme` is `'light'` or `'dark'`, kept in `localStorage` under `waqtuk.theme`. `themeClass` is the string `'wq-shell'` or `'wq-shell wq-dark'`; `AppShell` and `AuthLayout` put it on their root element and nothing else does.

Hook	Owner	Returns
`useToast()`	Dev 1	`{ show(message, tone = 'success') }`, `tone` is `'success'`, `'error'` or `'info'`. One toast at a time, it disappears after 3000 ms.
`useData()`	Dev 2	Section 4. `currentUser` inside it is always `users.find(u => u.id === useAuth().userId)`, so there is one source of truth for who is signed in.

Sign in flow (Login page, Dev 4). `const res = login(email, password)`. When `res.ok` the page calls `signIn(res.user.id)` and then navigates to the home route of the role: student `/dashboard`, mentor `/mentor`, admin `/admin`. `register(data)` returns the same shape and then the page signs the new student in the same way. **Seed accounts:** `student@example.com`, `mentor@example.com`, `admin@example.com`, all with the password `Password1`. The seed users also carry a `password` text field (mock data only, never shown on screen).

Route guard (Dev 4, inside `routes.jsx`). Not signed in: redirect to `/`. Signed in with the wrong role: redirect to the home route of the right role. A suspended user cannot sign in (the login action returns the error).

Every context file has the same three parts (Dev 4 writes `AuthContext`, Dev 1 writes `ThemeContext` and `ToastContext`, Dev 2 writes `DataContext`): a context, a provider that holds the state with `useState`, and a hook that components call. Example for theme:

```
// context/ThemeContext.jsx
import { createContext, useContext, useState, useEffect } from 'react';

const ThemeContext = createContext(null);

export function ThemeProvider({ children }) {
  const [theme, setTheme] = useState(() =>
localStorage.getItem('waqtuk.theme') || 'light');
  useEffect(() => { localStorage.setItem('waqtuk.theme', theme); }, [theme]);
  const value = {
    theme,
    setTheme,
    toggleTheme: () => setTheme(t => (t === 'light' ? 'dark' : 'light')),
    themeClass: theme === 'dark' ? 'wq-shell wq-dark' : 'wq-shell',
  };
  return <ThemeContext.Provider value={value}>{children}
</ThemeContext.Provider>;
}
```

```
export function useTheme() {
  return useContext(ThemeContext);
}
```

5.3 Layouts

Owners: **Dev 2** writes `Sidebar.jsx`, `navConfig.js` and `AuthLayout.jsx`; **Dev 3** writes `Topbar.jsx` and `AppShell.jsx`. `AppShell` draws the sidebar and the topbar and renders the page through `<Outlet />`. `AuthLayout` centres the login card. A page never draws a sidebar or a topbar itself. The topbar holds: menu button (mobile only), theme toggle, bell (opens the announcements of the role), account menu (Profile, Sign out). `navConfig.js` exports `NAV = { student: [...], mentor: [...], admin: [...] }`, each item `{ label, to, icon }`.

5.4 UI components

All components accept `className` and pass the rest of the props to the root element. All inputs are controlled: the page owns the value. Heights: Button, Input, Select 44 px (`size="sm"` is 36 px). Radius: 10 px controls, 16 px cards (section 3). Each component is `Name.jsx` plus `Name.css` (HTML structure, CSS classes starting with `wq-`, and the JavaScript for its props and events), so every owner writes all three languages in every component.

Dev	Components
Dev 1	`Badge`, `CategoryDot`, `ProgressBar`, `Toast`
Dev 2	`Button`, `Input`, `Select`, `Textarea`, `Checkbox`, `Radio`, `Icon`, `Card`, `Alert`, `PageHeader`
Dev 3	`Modal`, `ConfirmDialog`, `SegmentedControl`, `Table`
Dev 4	`StatCard`, `EmptyState`

`Icon` takes `name` and `size` (20) and draws the inline SVG with the same name from the prototype. `Toast` is the box that `ToastContext` shows.

Component	Props (default in brackets)
`Button`	`variant` `'primary'` | `'secondary'` | `'ghost'` | `'danger'` (`'primary'`), `size` `'md'` | `'sm'` (`'md'`), `type` (`'button'`), `disabled`, `loading` (shows a spinner and disables), `fullWidth`, `iconLeft` (node), `to` (a router path: renders a link that looks like the button), `onClick`, `children`
`Input`	`label`, `name`, `type` (`'text'`), `value`, `onChange(event)`, `placeholder`, `error` (text under the field, red), `hint` (text under the field), `required`, `disabled`, `iconLeft`
`Select`	`label`, `name`, `value`, `onChange(event)`, `options` `[{ value, label }]`, `placeholder`, `error`, `disabled`
`Textarea`	`label`, `name`, `value`, `onChange(event)`, `rows` (3), `maxLength`, `placeholder`, `error`, `hint`
`Checkbox`	`label`, `name`, `checked`, `onChange(event)`, `disabled`
`Radio`	`label`, `name`, `value`, `checked`, `onChange(event)`, `disabled` (one radio; the page draws several)
`Card`	`title`, `action` (node at the right of the title), `padding` `'md'` | `'lg'` | `'none'` (`'md'`), `children`
`Badge`	`tone` `'neutral'` | `'success'` | `'warning'` | `'danger'` | `'info'` (`'neutral'`), `children`
`Alert`	`tone` `'info'` | `'success'` | `'warning'` | `'danger'` (`'info'`), `title`, `onClose`, `children`
`Table`	`columns` `[{ key, header, render(row), align, hideOnSmall }]`, `rows`, `rowKey` (`'id'`), `emptyText`, `onRowClick(row)`. Without `render` it prints `row[key]`. `hideOnSmall` hides the column under 820 px.

Component	Props (default in brackets)
`Modal`	`open`, `title`, `onClose`, `footer` (node, usually the buttons), `size` `'md'` | `'sm'` (`'md'`), `children`. Closes on Esc, on the dark backdrop and on the X. Locks page scroll while open. Focus goes to the first field when it opens.
`ConfirmDialog`	`open`, `title`, `message`, `confirmLabel` (`'Confirm'`), `cancelLabel` (`'Cancel'`), `tone` `'default'` | `'danger'` (`'default'`), `onConfirm`, `onCancel`
`SegmentedControl`	`options` `[{ value, label }]`, `value`, `onChange(value)`. **Receives the value, not the event.**
`ProgressBar`	`value`, `max`, `tone` `'primary'` | `'success'` | `'warning'` | `'danger'` (`'primary'`), `label` (hidden text for screen readers). Percentage is clamped between 0 and 100.
`StatCard`	`label`, `value`, `hint`, `tone` (same tones as Badge, colours the hint)
`EmptyState`	`title`, `message`, `action` (node, usually a Button)
`CategoryDot`	`color`, `size` (10)
`PageHeader`	`title`, `subtitle`, `actions` (node at the right: buttons). Every page starts with it.

5.5 Charts

Owners: `BarChart` is Dev 1's, `DonutChart` is Dev 4's. Both charts are hand-written SVG, no chart library. Sizes follow the container width (`width: 100%`), the numbers below are the height.

Component	Props
`DonutChart`	`segments` `[{ id, label, minutes, color }]`, `centerValue` (text in the middle, e.g. the total), `centerLabel` (small text under it), `showLegend` (true). The legend shows the label, `formatDuration(minutes)` and the whole-number percentage from `percentages()` so the parts always add to 100. Segments with 0 minutes are not drawn.
`BarChart`	`data` `[{ key, label, minutes }]`, `height` (200), `highlightMax` (true: the tallest bar uses the strong colour, the others the soft colour), `ariaLabel`. Bars draw on the 0 baseline, the value shows with `formatDuration`.

5.6 Helper functions

All are plain functions, no React. Owners: `utils/time.js` and `utils/stats.js` Dev 1, `utils/ids.js` Dev 2, `utils/goals.js` Dev 3, the validator files each by the owner named in section 2. Dates are `'YYYY-MM-DD'`, date and time `'YYYY-MM-DDTHH:mm'`, durations whole minutes. An activity belongs fully to the day it starts (an overnight activity is not split at midnight). Ranges `{ from, to }` include both days.

```
// utils/time.js  (Dev 1)
export const TODAY = '2026-11-07';
formatDuration(min)            // 2040 -> '34h 00m', 150 -> '2h 30m', 45 ->
'0h 45m'
formatHours(min)               // 360 -> '6h', 330 -> '5.5h'
durationMinutes(start, end)    // minutes between two date-times
addDays(date, n)               // addDays('2026-11-07', -1) -> '2026-11-06'
startOfWeek(date)              // Monday of that week
endOfWeek(date)                // Sunday of that week
formatDateLong(date)           // 'Sat 7 Nov 2026'
formatDateShort(date)          // '7 Nov'
formatTime(dateTime)           // '09:00'
formatActivityRange(start, end)// '09:00–10:30'; an overnight activity prints
'23:00–02:00'
isFuture(dateTime)             // true when its date is after TODAY
getPeriodRange(period, offset = 0)
   // period 'day' | 'week' | 'month'; offset 0 current, -1 previous.
   // week: Monday to Sunday. month with offset 0: first day of the month to
TODAY.
```

```
   // returns { from, to, label }
getPreviousRange(period, offset = 0)
   // the range to compare with: day -> the day before, week -> the week
before,
   // month -> the same number of days of the month before

// utils/stats.js  (Dev 1)
sumByCategory(activities, from, to)      // { 'cat-study': 360, ... } by
start date
totalMinutes(activities, from, to)
percentages(values)                      // [360, 510, ...] -> whole numbers
adding to 100
                                          // (largest remainder method)
topCategoryId(totals, categories)        // largest total; a tie goes to the
first in category order
compare(current, previous)               // { diffMinutes, percent }  percent
rounded to a whole number

// utils/ids.js  (Dev 2)
makeId(prefix)                           // makeId('a') -> 'a-k3f9x2' (prefix
+ random text, unique per call)

// utils/goals.js  (Dev 3)
goalProgress(goal, loggedMinutes)
   // -> { logged, target, percent, status, diffMinutes }
   //  atLeast: logged >= target -> 'reached'; otherwise 'inProgress' while
the goal's
   //           week has not ended (weekStart + 6 >= TODAY), else 'missed'
   //  atMost:  logged <= target -> 'withinLimit'; otherwise 'exceeded'
   //  diffMinutes: |target - logged|
isOnTrack(status)                        // true for 'reached', 'inProgress',
'withinLimit'
goalSummary(goals, totalsByCategory)     // { onTrack: 3, total: 4 }
goalMessage(goal, loggedMinutes)         // 'Remaining 4h', 'Reached', ...
(see below)

// utils/validators/  one file per owner, index.js re-exports all of them and
merges MESSAGES
// every function returns { ok: true } or { ok: false, error: 'exact message'
}
// activity.js (Dev 1): validateActivity(a, { activities, categories })
// category.js (Dev 2): validateCategoryName(name, { categories, ownerId,
exceptId })
// goal.js     (Dev 3): validateGoal(g, { goals })
// account.js  (Dev 4): validateEmail(email), validatePassword(password) //
at least 8 characters
```

```
//                      validateRegistration(data, { users })
// each file also exports its own `messages` object; index.js exports
MESSAGES = { ...all of them }
// every error text is copied from the prototype states pages
```

The actions in `context/actions/*` call the validators and return their result. A page may call a validator itself for live checking while typing, but a page never writes its own copy of an error message: it shows `res.error` or `MESSAGES.xxx`.

Two additions to the list above. (1) `goalMessage(goal, loggedMinutes)` in `utils/goals.js` returns the exact text shown by the Goals page and by the Reports goals table: `'Remaining 4h'`, `'Reached'`, `'Allowance remaining 3h'`, `'Exceeded by 2h'`. The hours use `formatHours`, so a difference of 90 minutes prints `1.5h`. A missed goal prints the same Remaining text. The goal rows also show `Logged 6h of 10h`: both numbers use `formatHours`. (2) In `percentages()` and `topCategoryId()` a tie goes to the item that comes first in the list. With the week totals this gives Study 18, Work 25, Exercise 16, Sleep 9, Social 15, Gaming 17 (Study and Gaming have the same remainder, Study wins).

5.7 How every page is built

```
export default function Activities() {
  const { activities, categories, addActivity } = useData();   // 1. read
data and actions
  const toast = useToast();
  const [open, setOpen] = useState(false);                        // 2.
dialogs: one boolean each
  const [error, setError] = useState('');

  useEffect(() => { document.title = 'Activities · Waqtuk'; }, []);   // 3.
tab title (useEffect)

  function handleSave(values) {                                   // 4. every
action: call, check ok
    const res = addActivity(values);
    if (!res.ok) { setError(res.error); return; }
    setOpen(false);
    toast.show('Activity added');
  }

  return (
    <>
      <PageHeader title="Activities" actions={<Button onClick={() =>
setOpen(true)}>Add activity</Button>} />
      {/* cards, tables, charts */}
```

```
      <Modal open={open} title="Add activity" onClose={() =>
setOpen(false)}>...</Modal>
    </>
  );
}
```

- Each page also handles the states of its prototype states page: empty (an `EmptyState`) and error (an `Alert` or a field `error`). There is no loading state because the data is local.
- Colours, spacing and sizes come only from the CSS variables and classes of section 3. No hard-coded colours in a page.
- A page stores nothing in `localStorage`. Only `AuthContext`, `ThemeContext` and `DataContext` do.

6. Task sheet: Dev 1 (activities, calendar, dashboard, reports)

You own: the pages Activities (with `ActivityFormModal`), Dashboard, Calendar, Reports and AdminDashboard; the components `Badge`, `CategoryDot`, `ProgressBar`, `Toast` and `BarChart`; the logic `utils/time.js`, `utils/stats.js`, `utils/validators/activity.js`, `data/activities.json`, `context/actions/activities.js`, `context/selectors/activities.js`; the contexts `ThemeContext.jsx` and `ToastContext.jsx`; and the styling foundation `styles/theme.css` and `styles/global.css`. Every page and component is its own `.jsx` + `.css` pair.

Your three languages. HTML: the markup of 5 pages and 5 components. CSS: the whole theme (light and dark variables, the layout classes of the prototype) plus the CSS of every page and component you own. JavaScript: dates and statistics functions, the activity actions and selectors, two contexts, and the state and events of your pages.

Phase 0 (first hour): read this document, then create the branch `dev1/foundation`. Push `theme.css` and `global.css` first, within the first hour: all CSS variables of section 3 in two blocks, `.wq-shell` (light) and `.wq-shell.wq-dark` (dark), and in `global.css` the reset, the Inter font and the shared classes of the prototype (`.wq-g`, `.wq-hide-sm`, `.wq-main-pad`, `.wq-side`, `.wq-nav`). Everybody's CSS depends on these two files.

Phase 1 (core, about 2 days), in this order. Push after every step:

1. `ThemeContext.jsx` and `ToastContext.jsx` (use the template of section 5.2) and the `Toast` component.
2. `utils/time.js` and `utils/stats.js`: exactly the signatures of section 5.6. Check in the browser console that `formatDuration(2040)` gives `'34h 00m'` and that `percentages` of the week totals gives 18, 25, 16, 9, 15, 17.

3. `Badge`, `CategoryDot`, `ProgressBar`: markup, CSS and props exactly as section 5.4.
4. The activity data layer: `activities.json` (the seed of section 4, with the extra activities that make the totals come out exactly), `actions/activities.js`, `selectors/activities.js`, `validators/activity.js` (templates in section 4.1, texts from the prototype states pages).
5. `BarChart` (hand-written SVG, section 5.5).

Phase 2 (pages), in this order: Activities with `ActivityFormModal` first (Dashboard and Calendar reuse the modal), then Dashboard, Calendar, Reports, AdminDashboard. Details of each page are in section 10. Fixed decisions that are yours: the dashboard period controls are one `SegmentedControl` (Day, Week, Month) plus previous and next buttons that change `offset` (the next button is disabled at offset 0); the Activities page shows 5 rows and a "Show more" button; the calendar is a month grid, Monday first, the selected day defaults to `TODAY`, days with activities show a dot, and an overnight activity appears on both days (its minutes still count on its start day only); Reports export is the browser print dialog (`window.print()`) with an `@media print` block in `Reports.css` that hides the sidebar, topbar and buttons (no PDF library is installed).

You provide: the theme and toast to everybody, `time.js` and `stats.js` to everybody, the activity data to the dashboards and to the mentor pages, `ActivityFormModal`. **You consume:** `Button`, `Input`, `Select`, `Card`, `PageHeader`, `Modal`, `Table`, `SegmentedControl`, `StatCard`, `EmptyState`, `DonutChart`, `useData()`, `useAuth()`.

Done when: you can add, edit and delete an activity and the Dashboard, Calendar and Reports numbers change at once; the week shows 34h 00m with the donut percentages 18, 25, 16, 9, 15, 17; light and dark both work on your pages; Print on Reports shows only the report; your five pages match the prototype and its states pages.

7. Task sheet: Dev 2 (categories, announcements, profiles and the data hub)

You own: the pages Categories, AdminCategories, MentorAnnouncements, AdminAnnouncements, Profile and MentorProfile; the components `Button`, `Input`, `Select`, `Textarea`, `Checkbox`, `Radio`, `Icon`, `Card`, `Alert`, `PageHeader`; the layout pieces `Sidebar.jsx`, `navConfig.js`, `AuthLayout.jsx`; the logic `context/DataContext.jsx`, `utils/ids.js`, `utils/validators/category.js`, `data/categories.json`, `data/announcements.json`, `context/actions/categories.js`, `context/actions/announcements.js`, `context/selectors/catalog.js`; and the development-only preview page `pages/dev/ComponentsPreview.jsx`.

Your three languages. HTML: 6 pages, 10 components, the sidebar and the auth layout. CSS: the form controls (the biggest CSS job: sizes, focus rings, error states, light and dark) and the sidebar. JavaScript: the `DataContext` hub, id generation, category and announcement logic, and the editing state of your pages.

Phase 0: wait for Dev 4's skeleton (about 30 minutes), then create the branch `dev2/data-and-forms`. Read the prototype pages of your area and the sections 4 and 5 while you wait. Your first push (within about 2 hours) is **`DataContext` wired as in section 4.1**, so every page can call `useData()`; the actions of the other developers are placeholders until they push.

Phase 1 (core, about 2 days), in this order. Push after every step and never change a prop name after the first push:

1. `DataContext.jsx` and `utils/ids.js` (first push).
2. `Button`, `Input`, `Select`, `Textarea`, `Card`, `PageHeader` (everybody needs these first), then `Checkbox`, `Radio`, `Alert`, `Icon`. Each is `Name.jsx` plus `Name.css`, class names start with `wq-` (for example `wq-btn`, `wq-btn--danger`), styling only from the CSS variables of section 3. Every component works with the keyboard and has a visible focus ring.
3. `Sidebar.jsx`, `AuthLayout.jsx` and `navConfig.js` exactly as the prototype sidebars. Student: Dashboard `/dashboard`, Activities `/activities`, Calendar `/calendar`, Reports `/reports`, Goals `/goals`, Categories `/categories`, Sharing `/sharing`, Feedback `/feedback`, Profile `/profile`. Mentor: Home `/mentor`, Sharing Requests `/mentor/requests`, Shared Students `/mentor/students`, Announcements `/mentor/announcements`, Profile `/mentor/profile`. Admin: Dashboard `/admin`, Users `/admin/users`, Default Categories `/admin/categories`, Reported Content `/admin/reports`, Announcements `/admin/announcements`. The sidebar reads `NAV[role]` and highlights the active route. Check the labels and order against the prototype before the push.
4. The category and announcement data layer: the two JSON seeds of section 4, `actions/categories.js`, `actions/announcements.js`, `selectors/catalog.js`, `validators/category.js`.
5. `pages/dev/ComponentsPreview.jsx` (route `/components`, registered by Dev 4): every component in every variant, light and dark, so you check them without waiting for pages.

Phase 2 (pages), in this order: Categories, AdminCategories, MentorAnnouncements, AdminAnnouncements, Profile, MentorProfile. Details of each page are in section 10. Fixed decisions that are yours: the Profile page (student) has the profile card with name and email editable and the role shown read-only ("Your role cannot be changed."), the Save Changes button (calls `updateProfile`, Dev 4's action), the Log Out button, and the

announcements list with the detail panel on the right in a wrapper element with `id="announcements"`: the first announcement is selected at the start and clicking another one shows its title, publication date and text (list from `announcementsFor('students')`). `MentorProfile` is the same pattern for the mentor, without announcements. A link with a hash (the bell goes to `/profile#announcements`) scrolls to the element with that id: Dev 3 adds that small `useEffect` to `AppShell`.

You provide: `useData()` and the data hub to everybody, the form controls to every page, the sidebar. **You consume:** `Toast` and the theme (Dev 1), `Modal`, `Table`, `ConfirmDialog` (Dev 3), `StatCard`, `EmptyState` (Dev 4), `useAuth()`.

Done when: reloading the browser keeps every change and `resetData()` brings the seed back; the preview page shows every component in light and dark and every prop of sections 5.4 works; the sidebar shows the right items for each of the three roles; your six pages match the prototype and its states pages.

8. Task sheet: Dev 3 (goals, feedback, moderation and the dialogs)

You own: the pages Goals, Feedback, MentorStudent, MentorGoal and AdminReports; the components `Modal`, `ConfirmDialog`, `SegmentedControl`, `Table`; the layout pieces `Topbar.jsx` and `AppShell.jsx`; the logic `utils/goals.js`, `utils/validators/goal.js`, `data/goals.json`, `data/feedback.json`, `data/reports.json`, `context/actions/goals.js`, `context/actions/feedback.js`, `context/selectors/goals.js`.

Your three languages. HTML: 5 pages, 4 big components, the topbar and the app shell. CSS: the dialog layer (backdrop, scroll lock, focus), the table with its small-screen behaviour, the topbar and the responsive shell (sidebar drawer under 820 px). JavaScript: goal progress and messages, feedback and report workflow, focus handling and keyboard events in `Modal`.

Phase 0: wait for Dev 4's skeleton (about 30 minutes), then create the branch `dev3/dialogs-and-goals`. While you wait, read the prototype pages of your area and the sections 3 and 5.

Phase 1 (core, about 2 days), in this order. Push after every step and never change a prop name after the first push:

1. `Modal` and `ConfirmDialog` (closes on Esc, on the backdrop and on the X; locks page scroll; focus goes to the first field). Everybody's forms wait for these, so they are first.
2. `Table` and `SegmentedControl` (props of section 5.4; `Table` hides `hideOnSmall` columns under 820 px).

3. `AppShell.jsx` and `Topbar.jsx`: sidebar 240 px, topbar 64 px, content max width and padding as in the prototype; the sidebar becomes a drawer under 820 px opened by the menu button; `themeClass` from `useTheme()` goes on the root. The topbar holds: menu button (mobile only), theme toggle, bell (goes to `/profile#announcements` for a student, `/mentor/announcements` for a mentor, `/admin/announcements` for an administrator), account menu (Profile, Sign out). A link with a hash scrolls to the element with that id after navigation (a small `useEffect` in `AppShell`).
4. `utils/goals.js`: `goalProgress`, `isOnTrack`, `goalSummary`, `goalMessage` exactly as section 5.6. Check in the browser console that the seed goals give 3 of 4 on track.
5. The goal and feedback data layer: `goals.json`, `feedback.json`, `reports.json`, `actions/goals.js`, `actions/feedback.js`, `selectors/goals.js`, `validators/goal.js` (templates in section 4.1). You also create the two extra goals that the content reports of section 4 need.

Phase 2 (pages), in this order: Goals, Feedback, AdminReports, MentorStudent, MentorGoal. Details of each page are in section 10. Fixed decisions that are yours: the route `:id` of `MentorStudent` is a sharing request id and the route `:id` of `MentorGoal` is a goal id; both pages read data only through `getSharedData(requestId)` and `sharedStudents` (Dev 4), never from the raw lists; a mentor sees only the goals in the accepted request.

You provide: the dialogs, the table, the segmented control and the whole shell to everybody; goal logic to the Goals, Reports and mentor pages. **You consume:** `time.js` and `stats.js` (Dev 1), form controls and `Sidebar` (Dev 2), `getSharedData` and the sharing data (Dev 4), `useData()`.

Done when: every dialog opens and closes with the button, Esc and the backdrop; the app shell works at 1440 px and 390 px in light and dark; the Goals page shows 3 of 4 on track with the exact messages of section 5.6; a goal report reaches the administrator and the decision shows on the Feedback page; your five pages match the prototype and its states pages.

9. Task sheet: Dev 4 (skeleton, accounts, sharing and the people pages)

You own: the repository and the skeleton; `package.json`, `index.html`, `.gitignore`, `main.jsx`, `App.jsx`, `routes.jsx`, `context/AuthContext.jsx`; the pages Login, Sharing, MentorHome, MentorRequests, MentorStudents and AdminUsers; the components `StatCard`, `EmptyState` and `DonutChart`; the logic `data/users.json`,

`data/sharing.json`, `data/usage.json`, `utils/validators/account.js`, `context/actions/accounts.js`, `context/actions/sharing.js`, `context/selectors/accounts.js`.

Your three languages. HTML: 6 pages, 2 components and the page of every route. CSS: the login page, the request cards and the user table styling, `StatCard` and `EmptyState`. JavaScript: the whole skeleton and the routing, the sign-in flow and the route guard, account and sharing actions, the access rule for mentors and the donut chart geometry.

Phase 0 (first 30 minutes, everyone else is waiting for this):

1. Create the GitHub repository `waqtuk`. Run `npm create vite@latest waqtuk -- --template react`, then `npm install react-router-dom@6` (version 6, the one the course teaches). Delete the Vite demo files. Add `.gitignore` (`node_modules`, `dist`, `.env`, `.DS_Store`). Put the prototype site folder in `design/`.
2. Create **every file of the tree in section 2** as a placeholder with the right name and the right export: contexts export their provider and hook; each page is `export default function Name() { return <h1>Name</h1>; }`; each UI component renders a plain HTML element with the agreed props; each action is `(data) => ({ data, result: { ok: true } })`; each selector returns `[]`, `undefined` or a function; each validator returns `{ ok: true }`; each JSON seed is `[]` (`usage.json` is `{}`); each `index.js` is `export * from './File'` for every file of its folder. This is what lets everyone import real names from minute 30. After this push **nobody edits the `index.js` files again**.
3. Write `routes.jsx` with the 22 routes of section 2, the route guard of section 5.2, a catch-all route that redirects to `/`, and a development-only route `/components` (shown only when `import.meta.env.DEV` is true) pointing to Dev 2's `ComponentsPreview`. Write `main.jsx` with the provider order of section 5.2.
4. Push to `main` (the only direct push allowed in the whole project), then message the group: "skeleton is up, create your branch".

Phase 1 (core, about 2 days), in this order. Push after every step:

1. `AuthContext.jsx` (template of section 5.2: `userId` and `role` kept in `localStorage` under `waqtuk.session`) and the route guard helper it exports.
2. `StatCard` and `EmptyState` (section 5.4), then `DonutChart` (hand-written SVG, section 5.5; the legend percentages come from `percentages()`).
3. The account and sharing data layer: `users.json` (the three seed accounts and the students of section 4), `sharing.json`, `usage.json`, `validators/account.js`, `actions/accounts.js`, `actions/sharing.js`, `selectors/accounts.js` with the access rule of section 4.2 and `getSharedData(requestId)` (templates in section 4.1).

Phase 2 (pages), in this order: Login first (everybody tests with it), then Sharing, MentorHome, MentorRequests, MentorStudents, AdminUsers. Details of each page are in section 10. Fixed decisions that are yours: `LoginPage` is exactly the prototype `index.html`: one page with the Log in card (email, password, Log In button) and the Create an account card (Register as: Student or Academic Mentor, full name, email, password, confirm password, Create Account button, and the note that administrator accounts cannot be registered here). Log in calls `login`, then `signIn` and navigates to the home of the role (section 5.2). Create Account calls `register`, signs the new user in and navigates to their home. The "Preview the other roles" buttons (Mentor view, Administrator view) sign in as the seed mentor `m1` and the seed admin `a1` without a password. Errors show next to the field or in an `Alert` exactly as in the prototype `LoginStates.html`: use the `res.error` text and never your own wording. A signed-in user who opens `/` is redirected to their home.

You provide: the skeleton, routing, sign-in and the access rule to everybody; `StatCard`, `EmptyState`, `DonutChart`. **You consume:** everything else. Because you are the first real user of everyone's placeholders, write down any missing prop or action as a pull request comment on the owner's branch the same day.

Done when: sign up creates a student who can sign in and lands on the dashboard; an unknown email or a wrong password shows the exact prototype error; a suspended user cannot sign in; the mentor accepts a request and the student sees it accepted after reload; the mentor sees only the dates and goals of the accepted request; your six pages match the prototype and its states pages.

10. Page by page: what every page must contain

Routes and owners are in section 2. For every page below, the **prototype page is the visual and text reference** (`design/Name.html`) and its **states page** (`design/NameStates.html`) lists the dialogs, validation messages and empty states to build. Toast texts are the ones the prototype shows after the same button. If this table and the prototype differ, the seed rule of section 4 decides data, and the prototype decides everything else.

The prototype is only a prototype. It is the reference for the look, the layout and the wording of the screens, and every developer uses the copy they already have (an older copy is fine, nobody needs to replace it). The real app is the website of this document, and **where the prototype and this document disagree, this document wins** on everything in the list below. For anything not in the list, the prototype wins.

- **Data and numbers.** The seed of section 4 decides every name, date, count and total. The real app shows a week total of 34h 00m, 3 of 4 goals on track, and donut percentages Study 18, Work 25, Exercise 16, Sleep 9, Social 15, Gaming 17 (Study 18% and Gaming 17% even if an older copy shows them swapped).
- **The overnight Sleep activity** on the Activities page reads `23:00–02:00` (an older copy shows `23:00–07:00`, which contradicts the 3h of Sleep in the week).
- **Contradictions inside the prototype** (a student shown both pending and accepted, a suspended student shown as accepted, different wording or e-mails for the same thing): the seed rule of section 4 decides.
- **Time rules.** "Today" is always 2026-11-07, weeks run Monday to Sunday, durations are whole minutes, an activity counts fully on the day it starts, percentages use the largest remainder method.
- **Real behaviour the static prototype cannot show:** everything is stored and kept after a browser reload (`localStorage`); sign in, sign out and the route guard per role; a suspended user cannot sign in; a mentor sees only what an accepted request allows (section 4.2); every form validates with the messages of section 5.6 and shows them on the field; every button, dialog and link works; the Dashboard period control, the Reports print export and the calendar behave as the task sheets of sections 6 to 9 say.
- **Quality that a picture cannot show:** light and dark mode on every page, no horizontal scroll at 390 px width, keyboard use and visible focus on every control, no errors or warnings in the browser console, and no colour or size written by hand (only the variables of section 3).

10.1 Shared pieces that cross owners

Piece	Owner	Contract
`ActivityFormModal` in `pages/student/activities/ActivityFormModal.jsx`	Dev 1	Props `open`, `onClose`, `activity` (null to add, an activity to edit), `defaultDate` (`TODAY`). Fields: Category, Start date, Start time, End date, End time, Notes (optional); buttons Save Activity and Cancel. It calls `addActivity` or `updateActivity` itself, shows the error under the form, toasts on success and closes. Activities, Dashboard and Calendar are all Dev 1's pages and import it; nobody else builds an activity form. Dev 4's skeleton contains an empty placeholder of it, Dev 1 fills it in the first push of Phase 2.

Piece	Owner	Contract
`usageFor(from, to)` in `context/selectors/accounts.js` and `data/usage.json`	Dev 4	`usage.json` is the counts of section 4 for a 7 day range. `usageFor(from, to)` returns `{ students, mentors, activities, requests, reports }` equal to `Math.round(count * days / 7)`, so the default range (2 Nov – 8 Nov) shows exactly 14, 3, 412, 27, 5. Used by AdminDashboard (Dev 1).
Activity time text	Dev 1	`formatActivityRange(start, end)` in `utils/time.js` returns `'09:00–10:30'` (same day) or `'23:00–02:00'` (ends on another day); `formatDuration` gives the duration. Used by Activities, Calendar, Dashboard and the mentor pages.

10.2 Student pages

Page (owner)	Prototype files	Data and actions	Must build
LoginPage (Dev 4)	`index.html`, `LoginStates.html`	`login`, `register`, `signIn`	Section 9.

Page (owner)	Prototype files	Data and actions	Must build
Dashboard (Dev 1)	`Main.html` (and `Main.html?empty`)	`myActivities`, `myGoals`, `getPeriodRange`, `getPreviousRange`, `sumByCategory`, `totalMinutes`, `compare`, `topCategoryId`, `goalSummary`	Greeting by the hour: "Good morning" before 12:00, "Good afternoon" before 18:00, otherwise "Good evening", then the first name; "Add Activity" opens `ActivityFormModal`. Period control (Day, Week, Month) and previous and next buttons. Three stat cards: Total recorded time (with the change against the previous period), Largest category (name, time and share), Goal progress ("3 of 4 on track" and a `ProgressBar`). Cards "Time distribution" (`DonutChart`) and "Category totals" (`BarChart`, one bar per category). Card "Compared with the previous week" (day or month in the other periods). Goal progress uses the goals of the week that contains the last day of the shown period. Empty state "No activities recorded" when the period has none.

Page (owner)	Prototype files	Data and actions	Must build
Activities (Dev 1)	`Activities.html`, `ActivitiesStates.html`	`myActivities`, `myCategories`, `deleteActivity`	Card "Activity list" with filters Search (notes and category name), Category (All categories and each category), Date (All dates, Today, This week, This month). Table columns Date, Time, Category, Notes, Actions (Edit, Delete). Shows 5 rows with the text "Showing 5 of 17 activities" (`Showing {shown} of {total} activities`); a ghost button "Show more" adds 5 more rows. "Add Activity" and Edit open `ActivityFormModal` . Delete opens ConfirmDialog "Delete Activity?" with the prototype text. Empty and error states from the states page.

Page (owner)	Prototype files	Data and actions	Must build
Categories (Dev 2)	`Categories.html`, `CategoriesStates.html`	`categories`, `myCategories`, `addCategory`, `renameCategory`, `deleteCategory`	Card "Default categories" (read-only badge on each) and card "Custom categories" with table Name and Actions (Rename, Delete) and "Add Category". One modal "Add or rename category" (Category name, Save, Cancel). Delete dialog `Delete category "Reading"?` shows how many activities and goals use it and a required "Reassign to" select (all other usable categories); when nothing uses the category the dialog asks only for confirmation.

Page (owner)	Prototype files	Data and actions	Must build
Goals (Dev 3)	`Goals.html`, `GoalsStates.html`	`myGoals`, `myCategories`, `addGoal`, `updateGoal`, `deleteGoal`, `sumByCategory`, `goalProgress`, `goalMessage`	Week navigator (Previous week, "Week starting Mon 2 Nov 2026", Next week). Goal rows: category, At least or At most, Edit, Delete, "Logged 6h of 10h", the `goalMessage` text, a `ProgressBar`. "Add Goal" and Edit open a modal: Category (all usable categories), Week start date (any date, the page moves it to its Monday with `startOfWeek`), Target hours (hint "More than 0 and no more than 168 hours."), Direction (At least, At most), the tip about 56 hours of sleep, Save Goal, Cancel. The modal stores minutes (`hours * 60`). Delete dialog "Delete goal?" with the prototype text.

Page (owner)	Prototype files	Data and actions	Must build
Calendar (Dev 1)	`Calendar.html`, `CalendarStates.html`	`myActivities`, `deleteActivity`	Card "Activity calendar": Prev, month title, Next, Monday-first grid, a dot on days with activities, today and the selected day marked, legend ("Day with activities", "Selected date"). Selected day starts as `TODAY`. Panel with the selected date and its activities (category, time and duration, notes, Edit, Delete); an overnight activity also shows on the day it ends. Clicking an activity opens the "Activity details" panel (category, start, end, duration, notes). "Add Activity" and Edit use `ActivityFormModal` (`defaultDate` = selected day). Delete dialog "Delete Activity?" with the prototype text naming the activity and date.

Page (owner)	Prototype files	Data and actions	Must build
Reports (Dev 1)	`Reports.html`, `ReportsStates.html`	`myActivities`, `myGoals`, `sumByCategory`, `percentages`, `goalProgress`, `goalMessage`	Start date and End date (default: the current week) and Apply. "Export PDF" calls `window.print()`. Card "Category totals" (table Category, Time, Share, with a Total row of 100%) and card "Time distribution" (`DonutChart`, center total with "logged"). Card "Weekly goals in selected range": one row for each goal whose week starts inside the range (Goal with direction, "Week of Mon 2 Nov", Target, Actual with `formatHours`, Result with `goalMessage` in a `Badge`). Validation and empty states from the states page (for example end date before start date).

Page (owner)	Prototype files	Data and actions	Must build
Sharing (Dev 4)	`Sharing.html`, `SharingStates.html`	`users` (mentors), `myRequests`, `myGoals`, `sendRequest`, `cancelRequest`, `revokeAccess`	Card "New sharing request": Mentor (the three mentors), Start date, End date, Information to share (Summary, Reports), Goals to share (every goal of the week of the start date, with direction and target), the note about private notes, buttons "Review Scope" (shows a summary `Alert` of the choice) and "Send Request". Card "Existing access": one block per request of the student (mentor name, status `Badge`, approved range, information, goals) with "Cancel Pending Request" for pending and "Revoke Access" for accepted, plus the note "Changing the scope replaces all current access with a new pending request." Dialogs "Cancel pending request?" (Keep Request, Cancel Request) and "Revoke access?" (Keep Access, Revoke Access). One request per mentor: a new one for the same mentor replaces the old pending one only after the old one is cancelled (the action returns the error otherwise).

Page (owner)	Prototype files	Data and actions	Must build
Feedback (Dev 3)	`Feedback.html`, `FeedbackStates.html`	`myFeedback`, `getUser`, `reportFeedback`	Card "Feedback" with `SegmentedControl` (All, Comments, Suggestions, Flags) and the list (type badge, mentor name, date, text). Selecting a row shows the "Feedback detail" panel (type, "From Mentor A · 5 Nov 2026", `detail ?? text`, "Related goal: Study (at least 10h), week of Mon 2 Nov", the note "Advice never changes your records automatically. Update your goals yourself if you agree.", buttons "Report Content" (disabled when already reported) and "Open Goals" to `/goals`). Report dialog: Reason (Inappropriate language, Harassment, Spam, Other), Details (optional), Cancel, Submit Report. First row selected at the start.
Profile (Dev 2)	`Profile.html`, `ProfileStates.html`	`currentUser`, `updateProfile`, `announcementsFor('students')`	Section 7.

10.3 Mentor pages

Page (owner)	Prototype files	Data and actions	Must build
MentorHome (Dev 4)	`MentorHome.html`, `MentorHomeStates.html`	`mentorRequests`, `sharedStudents`, `announcementsFor('mentors')`, `acceptRequest`, `declineRequest`, `endSharing`	Three stat cards: Pending requests, Shared students, Announcements ("1 new": published in the 7 days up to `TODAY`). Card "Sharing requests" (first pending request in full with proposed range, data, goals, the note about what the mentor will see, Accept and Decline; the next one as "Next in queue"). Card "Shared students" (search by name, rows with status, selected row shows scope with "Open Overview" to `/mentor/students/:requestId` and "End Sharing"). Card "Announcements" (list and detail). Dialog "End sharing?" and the decline dialog from `MentorRequests`.
MentorRequests (Dev 4)	`MentorRequests.html`, `MentorRequestsStates.html`	`mentorRequests`, `acceptRequest`, `declineRequest`	Stat cards Pending, Accepted this term, Declined (counted from the requests). List of every pending request with Accept and Decline. Table "Recent decisions" (Student, Decision, Date from `decidedOn`, Scope). Dialog "Decline this request?" with the prototype text.
MentorStudents (Dev 4)	`MentorStudents.html`, `MentorStudentsStates.html`	`sharedStudents`, `endSharing`	Stat cards Shared students, Goals shared (sum of `goalIds`), Ending soon (accepted requests whose `to` is on or before `TODAY`, with the first student and date). Filters Search and Scope (All scopes, Summary, Reports). Table Student, Data shared, Approved period, Goals, Status, Actions (Open to `/mentor/students/:requestId`, End). Dialog "End sharing?".

Page (owner)	Prototype files	Data and actions	Must build
MentorStudent (Dev 3)	`MentorStudent.html`, `MentorStudentStates.html`	`getSharedData(requestId)`	`:id` in the route is the **request id**. Selector "Shared student" listing `sharedStudents` (changing it navigates). "Approved range" with View from, View to, Apply (the dates cannot leave the approved range). Scope note ("Approved scope: Summary, Reports and goal Study (at least 10h). Private notes are hidden."). Table "Category totals" and `DonutChart` for the chosen range. Card "Shared goal progress" (each shared goal with "Logged 6h of 10h · Remaining 4h" and the button "Give feedback on this goal" to `/mentor/goals/:goalId`). Table "Weekly reports" for the current week and the two before: covered dates are the overlap of the week with the approved range, "(full week)" when the overlap is the whole week, " (partial week)" otherwise, "Not in approved range" when empty; selecting a covered row shows Total time, Top category and goal progress of that week. Only the student's `mentorName` is shown.
MentorGoal (Dev 3)	`MentorGoal.html`, `MentorGoalStates.html`	`getSharedData`, `getUser`, `addFeedback`	`:id` is the **goal id**; if no accepted request of this mentor contains it, show the unavailable state of the states page. Header card (student, approved period, scope), card "Shared goal" with progress, card "Existing discussion" (this mentor's feedback on the goal, date and text), comment form (Comment, Post Comment), card "Guidance actions" with "Suggest Adjustment" (dialog: Advice, Cancel, Send) and "Flag Imbalance" (dialog: Explanation, Cancel, Submit Flag), and the two notes at the bottom. The three actions call `addFeedback` with type comment, suggestion, flag.
MentorAnnouncements (Dev 2)	`MentorAnnouncements.html`, `MentorAnnouncementsStates.html`	`announcementsFor('mentors')`	Card "Announcements for mentors": Search, list with a "New" badge (published in the 7 days up to `TODAY`), selected item detail (title, "Audience: All users · 5 Nov 2026", text). First row selected at the start.
MentorProfile (Dev 2)	`MentorProfile.html`, `MentorProfileStates.html`	`currentUser`, `updateProfile`	Section 7.

10.4 Administrator pages

Page (owner)	Prototype files	Data and actions	Must build
AdminDashboard (Dev 1)	`AdminDashboard.html`, `AdminDashboardStates.html`	`usageFor`	Start date, End date (default 2 Nov – 8 Nov), Apply. Five stat cards: New students, New mentors, Activities, Sharing requests, Reports, each with "in selected range". A card "Usage in selected range" listing the same five numbers inside a wrapper with `id="usage"`, and the card "Quick links" (Users, Default categories, Reported content, Announcements). Totals only, never private records.
AdminUsers (Dev 4)	`AdminUsers.html`, `AdminUsersStates.html`	`users`, `updateUser`, `setUserStatus`	Card "Accounts": Search (name or email), Role filter (All roles, Student, Mentor), Status filter (All statuses, Active, Suspended), table Name, Email, Role, Status, Action (Suspend or Reactivate). Selecting a row fills the card "Account details" (Name, Email, Role read-only, Status read-only, Save Changes). Status management asks for a required Reason; the dialog "Suspend this account?" shows the prototype text. Suspending a user also sets their pending requests to `'cancelled'` and accepted ones to `'revoked'` (done inside `setUserStatus`).
AdminCategories (Dev 2)	`AdminCategories.html`, `AdminCategoriesStates.html`	`categories`, `addDefaultCategory`, `renameDefaultCategory`, `setCategoryActive`	Table of default categories: Name, Status (Active or Inactive badge), Actions (Rename, Deactivate or Reactivate), "Add Category", the modal "Add or rename category", the note about inactive categories. Dialogs `Deactivate "Study"?` and `Reactivate "Commute"?` with the prototype texts.

Page (owner)	Prototype files	Data and actions	Must build
AdminReports (Dev 3)	`AdminReports.html`, `AdminReportsStates.html`	`contentReports`, `feedback`, `getUser`, `decideReport`	Card "Pending reports": Search, Filter (Pending, Closed), list ("Report 1: Mentor comment", reason, submitted date). Selecting one shows "Report detail": the reported text exactly as the student saw it (`detail ?? text`), reason, details, author, reporter, date, a required "Decision reason", buttons "Dismiss Report" and "Remove Content" with the note that removing also closes other pending reports for the same content. Dialog "Remove this content?" with the prototype text. `decideReport(..., 'removed', ...)` marks the feedback `removed: true` (hidden from every list) and closes the other pending reports of that feedback with the same decision.
AdminAnnouncements (Dev 2)	`AdminAnnouncements.html`, `AdminAnnouncementsStates.html`	`announcements`, `saveAnnouncement`, `publishAnnouncement`, `unpublishAnnouncement`	Table Title, Audience (Students, Mentors, All users), Status, Actions (Edit, then Publish Draft for drafts or Unpublish for published ones, nothing else for unpublished), the note about drafts. Card "Announcement": Title, Message, Audience, Save Draft, Publish, Cancel; Edit loads the row into it. Dialog "Unpublish announcement?" with the prototype text.

11. Git rules, merge order and the finish line

11.1 Who is who

Dev	Person	Branch prefix
Dev 1	Hammad Albalawi (team leader)	`dev1/`
Dev 2	Fahad Altuwayjiri	`dev2/`
Dev 3	Abdallah Al-Mogbel	`dev3/`
Dev 4	Saeed Ibrahim Alorayj	`dev4/`

The team leader decides every change to a contract (sections 2 to 5) and every dispute about who owns a file. Names can be swapped between the four rows; nothing else in this document changes.

11.2 Git rules (all fixed)

- **Setup.** Node 20 or newer. Commands: `npm install`, `npm run dev` (run), `npm run build` (must pass before every pull request). Only Dev 4 (the repository owner) installs packages; anybody who needs one asks in the group first.
- **Branches.** `main` only receives pull requests (the one exception is the skeleton of Phase 0). Branch name: `dev1/activities-page`, `dev2/data-context`, `dev3/dialogs`, `dev4/login-page`. One branch per task, one page or one component group per pull request, at most about 400 changed lines.
- **Identity.** Each person commits with their own GitHub account and the same e-mail as that account (`git config user.name` and `user.email`), otherwise the commits do not count in the contribution graph.
- **Commit messages.** English, present tense, one idea each, prefix with the type: `feat: add activity list table`, `fix: overlap check for overnight activities`, `style: dark mode colors for Badge`, `refactor:`, `chore:`. Commit at least once per working session, never one huge commit at the end.
- **Before a pull request.** `git fetch origin`, `git merge origin/main`, fix conflicts, run `npm run build`, open the page in light and dark and at 390 px width. The pull request text has three lines: what changed, which page or files, how you tested it, with screenshots for pages.
- **Review.** One other developer reviews each pull request the same day. The review checks only: matches the prototype, uses the components and helpers of section 5, no colours written by hand, no file of someone else edited, error texts come from

`MESSAGES`, no warnings in the browser console.

- **Merging.** Use "Create a merge commit" (not squash) so every commit stays in the history under its author. The author merges after the approval.
- **Conflicts.** A conflict in a file you own: solve it yourself. A conflict in a file you do not own means you edited it: undo your change in that file and ask its owner.
- **Locked files** (only the owner edits them): `package.json`, `routes.jsx`, `main.jsx`, `App.jsx` (Dev 4); `navConfig.js` (Dev 2); `DataContext.jsx` (Dev 2); every `index.js` of `context/actions/`, `context/selectors/`, `utils/validators/`, `components/ui/` and `components/charts/` (nobody edits them after Dev 4's skeleton, they only contain `export *` lines). A missing route, nav item, action, selector or export is requested in a pull request comment and the owner adds it the same day. A new action or selector is a function in your own file, so it needs no one else.
- **Changing a contract** (a prop, a function signature, a data shape): ask in the group, the team leader approves, this document is edited first, then the owner changes the code. Adding something new is always allowed; renaming or removing is not.

11.3 Merge order, so nobody waits

Step	What lands on main	Who	Unblocks
0	Skeleton with every file as a placeholder, 22 routes, providers	Dev 4 (direct push)	everybody
1	`theme.css`, `global.css`, `ThemeContext`, `ToastContext`	Dev 1	all styling, light and dark
1	`DataContext` with all actions present, `ids.js`	Dev 2	all pages
1	`Button`, `Input`, `Select`, `Textarea`, `Card`, `PageHeader` (form controls first)	Dev 2	all pages
1	`Modal`, `ConfirmDialog`, `Table`, `SegmentedControl`, `AppShell`, `Topbar`	Dev 3	all forms and navigation
1	`AuthContext`, route guard, `LoginPage`	Dev 4	sign in and navigation

Step	What lands on main	Who	Unblocks
2	`time.js`, `stats.js`, `goals.js`, `Badge`, `ProgressBar`, `CategoryDot`, `StatCard`, `EmptyState`, the other form controls, `Sidebar`	Dev 1, 2, 3, 4	Dashboard, Goals, Profile
3	`BarChart` (Dev 1) and `DonutChart` (Dev 4)	Dev 1, Dev 4	Dashboard, Reports, MentorStudent
4	The real logic inside each developer's own actions, selectors and validators (placeholders get replaced, signatures stay)	everybody	all form pages
5	Pages, each merged when it works	everybody	

While a piece you need is still a placeholder, build your page against the placeholder: the imports, the props and the layout are final, only the look or the numbers improve later. Never copy a component or a helper into your own folder as a temporary version.

11.4 A page is done when

1. It looks like its prototype page in light and dark, at about 1440 px and at 390 px width (no horizontal scroll).
2. Every button and link works; every dialog opens and closes (button, Esc, backdrop); every success shows the prototype toast; every error shows the exact text from `MESSAGES` or from `res.error` next to the field or in an `Alert`.
3. It uses only the components, helpers and CSS variables of this document and writes no colour or size by hand.
4. Its numbers equal the seed of section 4 (week total 34h 00m, 3 of 4 goals on track, percentages 18, 25, 16, 9, 15, 17).
5. Its empty and error states of the states page are built.
6. `npm run build` passes and the browser console has no errors or warnings.
7. A teammate reviewed it and it is merged into `main`.

11.5 The project is done when this click-through works (Phase 3, all four together)

- **Student (`student@example.com`).** Dashboard shows 34h 00m and "3 of 4 on track" and switches Day, Week, Month. Add an activity that overlaps another: the exact overlap error shows. Add one in the future: the exact error shows. Add a valid one: the

dashboard, Activities, Calendar, Reports and Goals numbers all change. Edit and delete an activity from Activities and from Calendar. Rename a custom category; delete `Reading`, reassigning its 4 activities and 1 goal. Add, edit, delete a goal (target 0 and 169 hours are rejected). Reports: change the range, export through the print dialog. Sharing: send a request to Mentor C, cancel it, revoke Mentor A. Feedback: filter, open one, report one. Profile: edit name and email (a used e-mail is rejected), read an announcement. Log out.

- **Mentor (`mentor@example.com`).** Accept Student D, decline Student F; both counters on Home and Sharing Requests change. Open Student A through Shared Students, change the range inside and outside the approved dates, open the goal, post a comment, a suggestion and a flag. End sharing with Student B. Read the announcements.
- **Administrator (`admin@example.com`).** Dashboard range changes the five numbers. Users: search, filter, edit an e-mail, suspend Student A with a reason and check that the student cannot sign in and that their requests became revoked or cancelled, then reactivate. Default categories: deactivate Study and check that the Activities form no longer offers it, then reactivate; add and rename one. Reported content: dismiss one, remove one (the feedback disappears for the student and the other reports for it close). Announcements: save a draft, publish it, and check that it appears in the student Profile or mentor Announcements; unpublish it and check that it disappears.
- **Cross effects that prove the data is shared:** the mentor's comment appears on the student's Feedback page; a student's report appears in the administrator's list; a published announcement reaches the right role; a deactivated category is not selectable; suspending a user ends their sharing.
- Light and dark and 390 px width checked on every page; `npm run build` passes on `main`.